Dual-Purpose Health Monitor

A Low-Cost Infant Incubator & Air Quality System

Eden Gilbert Kiseka & Okurut Joe

December 25, 2025
Version 1.1 – Kampala, Uganda

Abstract

This document serves as the complete technical manual for the Dual-Purpose Health Monitor. It details the system architecture, hardware requirements, assembly instructions, software implementation, and maintenance procedures for a low-cost, Arduino-based solution designed for resource-limited settings. The system operates in two modes: neonatal incubator monitoring and environmental air quality assessment.

1 Project Overview

This dual-purpose system addresses critical health monitoring needs in resource-limited settings. It is designed to be cost-effective, easily replicable, and simple to maintain.

1.1 Core Functions

1. **Mode 1: Infant Incubator Monitor** – Tracks temperature and humidity to ensure safe conditions for neonates.
2. **Mode 2: Indoor Air Quality Monitor** – Assesses environmental air quality using an AQI index.

1.2 Key Features

- Real-time sensing (Temperature, Humidity, Gas/Air Quality).
- Triple-display output (OLED for data, LCD for status, 7-Segment for quick values).
- Visual alert system with Red/Yellow/Green LEDs.
- User-adjustable thresholds via potentiometer.

2 System Architecture

The system architecture is centered around an Arduino Uno R3, acting as the primary controller. It interfaces with analog and digital sensors on the input side and manages multiple display protocols on the output side.

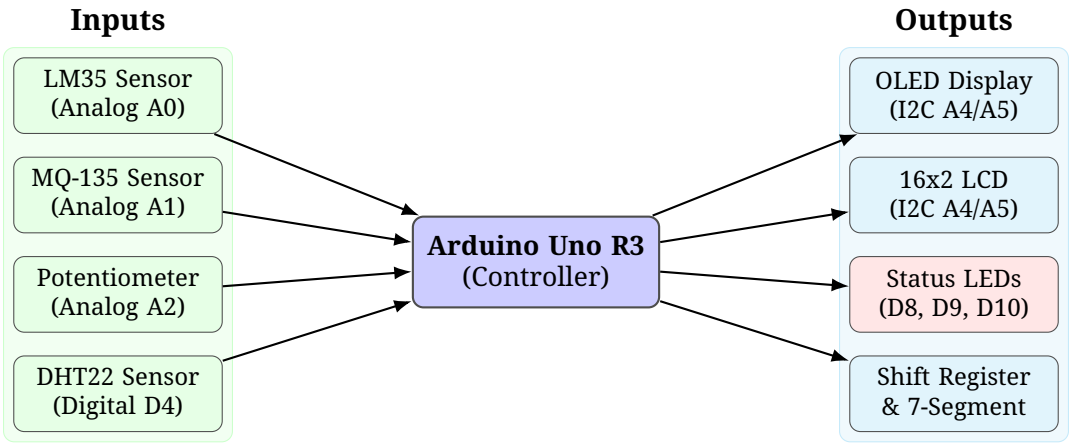

Figure 1: System Architecture Diagram showing Input and Output layers connected to the central controller.

3 Hardware Requirements

The Bill of Materials (BOM) is listed in Table 1. Prices are estimates based on local availability in Kampala.

Table 1: Component List

Component	Qty	Specifications
Arduino Uno R3	1	ATmega328P MCU
DHT22	1	Temp & Humidity
LM35	1	Analog Temp Sensor
MQ-135	1	Air Quality Sensor
OLED Display	1	0.96”, 128x64, I2C
LCD 16x2	1	I2C Backpack
7-Segment	1	Common Cathode, Red
74HC595	1	8-bit Shift Register
LEDs	3	R/Y/G (5mm)
Potentiometer	1	10kΩ Linear
Resistors	10+	220Ω, 10kΩ

4 Circuit Assembly

4.1 Pin Assignment Map

The system utilizes both analog and digital interfaces. The specific pin mapping is detailed in Table 2.

4.2 Step-by-Step Hardware Assembly

To ensure a safe and functional build, follow these steps in order. Disconnect all power sources before beginning.

Table 2: Pinout Configuration

Pin	Component	Function
A0	LM35	Temp Input
A1	MQ-135	Air Quality Input
A2	Potentiometer	Threshold Adj.
A4	OLED / LCD	I2C SDA
A5	OLED / LCD	I2C SCL
D4	DHT22	Digital Data
D8	Green LED	Safe Status
D9	Yellow LED	Warning Status
D10	Red LED	Danger Alert
D11	74HC595	Data (SER)
D12	74HC595	Clock (SRCLK)
D13	74HC595	Latch (RCLK)

4.2.1 Step 1: Power Distribution

1. Establish power rails on the breadboard.
2. Connect the Arduino **5V** pin to the red positive (+) rail.
3. Connect the Arduino **GND** pin to the blue negative (-) rail.
4. *Verification:* Use a multimeter to check for continuity and ensure no short circuits between rails.

4.2.2 Step 2: Sensor Connections

1. **LM35 (Temp):** Connect Pin 1 to 5V, Pin 3 to GND, and Pin 2 (Signal) to Arduino **A0**.
2. **DHT22 (Temp/Hum):** Connect Pin 1 to 5V, Pin 4 to GND, and Pin 2 (Data) to Arduino **D4**. *Crucial:* Place a 10kΩ resistor between Pin 1 and Pin 2 as a pull-up.
3. **MQ-135 (Air Quality):** Connect VCC to 5V, GND to GND, and Analog Output (A0) to Arduino **A1**. Note: This sensor requires a warm-up period.

4.2.3 Step 3: Display Connections

Both the OLED and LCD use the I2C protocol, allowing them to share pins.

1. Connect **SDA** on both displays to Arduino **A4**.
2. Connect **SCL** on both displays to Arduino **A5**.
3. Connect VCC to 5V and GND to GND for both units.

4.2.4 Step 4: Output Components

1. **Status LEDs:** Connect the Anode (long leg) of Green, Yellow, and Red LEDs to pins **D8**, **D9**, and **D10** respectively. Place a 220Ω resistor in series with each cathode to ground.
2. **7-Segment & Shift Register:**
    - Insert the 74HC595 chip. Connect Pin 16 (VCC) and 10 (SRCLR) to 5V. Connect Pin 8 (GND) and 13 (OE) to GND.
    - Connect Control Pins: SER to **D11**, SRCLK to **D12**, RCLK to **D13**.
    - Connect Outputs Q0-Q7 to the 7-segment display segments (A-G + DP) via 220Ω resistors.

4.2.5 Step 5: User Input

1. Place the 10kΩ potentiometer.
2. Connect the outer legs to 5V and GND.
3. Connect the center wiper pin to Arduino **A2**.

5 Software Configuration

5.1 Prerequisites

Ensure the Arduino IDE is installed on your computer.

5.2 Library Installation

Navigate to `Sketch` → `Include Library` → `Manage Libraries...` and install the following:

- `DHT sensor library` by Adafruit
- `Adafruit SSD1306`
- `Adafruit GFX Library`
- `LiquidCrystal I2C`

5.3 Firmware Upload

1. Connect the Arduino Uno to your PC via USB.
2. Select the correct board: `Tools` → `Board` → `Arduino Uno`.
3. Select the correct port: `Tools` → `Port`.
4. Copy the code provided in Appendix 1 into the editor.
5. Click **Upload**. Wait for the ”Done uploading” message.

6 Operating Instructions

6.1 Incubator Mode

This mode is optimized for monitoring neonatal environments.

- **Green LED:** Safe range (32–38°C).
- **Red LED:** Outside safe range.
- **Control:** Rotate potentiometer to shift the safe threshold center point.

6.2 Air Quality Mode

This mode assesses general indoor air quality (IAQ).

- **Scale:** 0–9 (Relative AQI).
- **Alerts:** Red LED triggers if AQI > 7.

7 Maintenance

Table 3: Maintenance Schedule

Interval	Action
Daily	Visual check of displays/LEDs.
Weekly	Clean sensors; verify thresholds.
Monthly	Compare temp with reference.
Annually	Replace MQ-135; Full Calibration.

8 Troubleshooting

Common issues often relate to power delivery or sensor connection. Figure 2 outlines the diagnostic process.

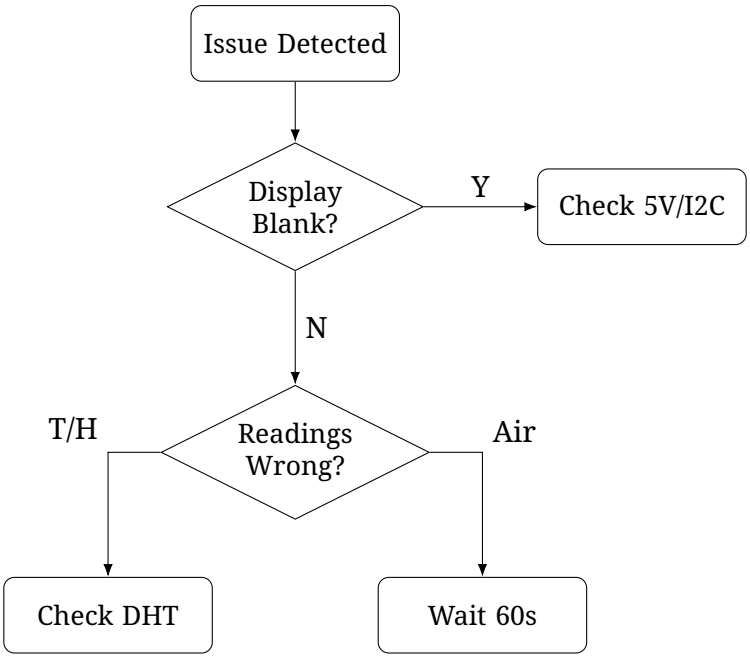

Figure 2: Diagnostic Flowchart

A Software Implementation

The following C++ code is designed for the Arduino IDE. It requires the `Adafruit_SSD1306`, `LiquidCrystal_I2C`, and `DHT` libraries.

```
#include <Wire.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

// --- PIN DEFINITIONS ---
#define PIN_LM35     A0
#define PIN_MQ135    A1
#define PIN_POT      A2
#define PIN_DHT      4
#define PIN_LED_G    8
#define PIN_LED_Y    9
#define PIN_LED_R    10
#define PIN_SER      11  // 74HC595 Data
#define PIN_SRCLK    12  // 74HC595 Clock
#define PIN_RCLK     13  // 74HC595 Latch

// --- CONSTANTS & OBJECTS ---
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET    -1
Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET);
LiquidCrystal_I2C lcd(0x27, 16, 2);
DHT dht(PIN_DHT, DHT22);

// 7-Segment Patterns (0-9)
const byte digitMap[] = {
  0b00111111, 0b00000110, 0b01011011, 0b01001111,
```

```
  0b01100110, 0b01101101, 0b01111101, 0b00000111,
  0b01111111, 0b01101111
};

void setup() {
  // Initialize Serial & Pins
  Serial.begin(9600);
  pinMode(PIN_LED_G, OUTPUT);
  pinMode(PIN_LED_Y, OUTPUT);
  pinMode(PIN_LED_R, OUTPUT);
  pinMode(PIN_SER, OUTPUT);
  pinMode(PIN_SRCLK, OUTPUT);
  pinMode(PIN_RCLK, OUTPUT);

  // Initialize Sensors & Displays
  dht.begin();
  lcd.init();
  lcd.backlight();

  if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
    Serial.println(F("SSD1306␣allocation␣failed"));
    for(;;);
  }
  display.display();
  delay(1000);
  display.clearDisplay();
}

void loop() {
  // 1. Read Sensors
  float tempDHT = dht.readTemperature();
  float humDHT = dht.readHumidity();
  int rawMQ = analogRead(PIN_MQ135);
  int rawLM35 = analogRead(PIN_LM35);
  float tempLM35 = (rawLM35 * 5.0 / 1024.0) * 100.0;

  // 2. Read User Threshold
  int thresholdRaw = analogRead(PIN_POT);
  int setPoint = map(thresholdRaw, 0, 1023, 30, 40); // 30-40C Range

  // 3. Logic & Alerts
  // Use DHT as primary, fallback to LM35
  float currentTemp = isnan(tempDHT) ? tempLM35 : tempDHT;

  if (currentTemp > setPoint + 1 || currentTemp < setPoint - 1) {
    digitalWrite(PIN_LED_G, LOW);
    digitalWrite(PIN_LED_Y, LOW);
    digitalWrite(PIN_LED_R, HIGH); // Danger
  } else {
    digitalWrite(PIN_LED_G, HIGH); // Safe
    digitalWrite(PIN_LED_Y, LOW);
    digitalWrite(PIN_LED_R, LOW);
  }

  // 4. Update Displays
  updateOLED(currentTemp, humDHT, rawMQ);
  updateLCD(currentTemp, setPoint);
  update7Seg((int)currentTemp);
```

```
88
89    delay(2000); // Wait 2s before next reading
90  }
91
92  void updateOLED(float t, float h, int air) {
93    display.clearDisplay();
94    display.setTextSize(1);
95    display.setTextColor(SSD1306_WHITE);
96    display.setCursor(0,0);
97    display.println(F("HEALTH␣MONITOR"));
98    display.setTextSize(1);
99    display.print(F("Temp:␣")); display.print(t); display.println(F("␣C"));
100   display.print(F("Hum:␣␣")); display.print(h); display.println(F("␣%"));
101   display.print(F("AQI:␣␣")); display.println(air);
102   display.display();
103 }
104
105 void updateLCD(float t, int sp) {
106   lcd.setCursor(0, 0);
107   lcd.print("Temp:␣"); lcd.print(t, 1); lcd.print("C");
108   lcd.setCursor(0, 1);
109   lcd.print("Set:␣␣"); lcd.print(sp); lcd.print("C");
110 }
111
112 void update7Seg(int num) {
113   // Display tens digit only (simplified)
114   int displayNum = (num / 10) % 10;
115   if (num < 10) displayNum = num;
116
117   digitalWrite(PIN_RCLK, LOW);
118   shiftOut(PIN_SER, PIN_SRCLK, MSBFIRST, digitMap[displayNum]);
119   digitalWrite(PIN_RCLK, HIGH);
120 }
```

Listing 1: Complete Arduino Firmware